
ICPC Algorithm Library

目 录		5 Geometry	13
1 Basic	1	5.1 vector	13
1.1 test	1	5.2 seg_poly	13
1.2 pai	1		
1.3 run	1		
1.4 缺省源	1		
1.5 随机数	1		
1.6 Matrix	1		
2 Graph	1		
2.1 Dijkstra	1		
2.2 Tarjan SCC	2		
2.3 2-SAT	2		
2.4 Segment Tree Graph	2		
3 Data Structure	3		
3.1 SGT	3		
3.2 LazySGT	4		
3.3 Chairman Tree	5		
3.4 Li Chao Tree	6		
3.5 Segment Tree Beats	7		
3.6 Segment Tree Merge / Split	8		
4 String	9		
4.1 KMP	9		
4.2 Z Function	10		
4.3 String Hash	10		
4.4 Manacher	10		
4.5 Aho-Corasick	11		
4.6 Suffix Array	11		
4.7 Suffix Automaton	12		
4.8 Palindromic Tree	12		
4.9 Lyndon / Minimum Representation .	13		

1 Basic

1.1 test

```
export fs="-fsanitize=address,undefined"
ulimit -s 1024000
ulimit -v 1024000
mk() { g++ -o $1 $1.cpp -O2; }
```

1.2 pai

```
pai() {
    mk a; mk bf; mk gen
    while true; do
        ./gen > 1.in
        ./bf < 1.in > 1.ans
        ./a < 1.in > 1.out
        if diff -Zq 1.out 1.ans; then echo ac; else echo wa; break; fi
    done
}
```

1.3 run

```
shopt -s nullglob # 没有匹配到任何文件时, 让它返回空
ulimit -s 1024000
g++ $1.cpp -o $1 -std=c++20 -O2 -fsanitize=address,undefined || exit
for f in "$2"/*.in; do
    x=${f%.in}
    \time -f "$x: %es %MKB" ./$1 < $x.in > $1.out
    diff -Zq $1.out $x.ans
done
```

1.4 缺省源

```
#include <bits/stdc++.h>
using namespace std;

#define ALL(s) s.begin(), s.end()
#define SZ(s) int(s.size())
#define pb push_back

using LL = long long;
using ULL = unsigned long long;
using I = __int128;

void Cas() {
}

int main() {
```

```
cin.tie(0)->sync_with_stdio(0);
int t = 1;
cin >> t;
while (t--) {
    Cas();
}
return 0;
}
```

1.5 随机数

```
ULL seed = chrono::steady_clock::now().time_since_epoch().count();
mt19937_64 rnd(seed);
```

1.6 Matrix

```
template <class T>
struct Mat {
    int n, m;
    vector<T> a;
    Mat(int n = 0, int m = 0, T v = {}) : n(n), m(m), a(n * m, v) {}
    T& operator()(int i, int j) { return a[i * m + j]; }
};

using M = Mat<LL>;
M mul(M a, M b) {
    M c(a.n, b.m);
    for (int i = 0; i < a.n; i++) {
        for (int j = 0; j < b.m; j++) {
            for (int k = 0; k < a.m; k++) {
                c(i, j) = max(c(i, j), a(i, k) + b(k, j));
            }
        }
    }
    return c;
}
```

2 Graph

2.1 Dijkstra

```
struct Node {
    int u;
    LL d;
    bool operator<(const Node& o) const {
        return d > o.d;
    }
};
```

```
priority_queue<Node> q;
vector<LL> D(n, INF);
D[0] = 0;
q.push({0, 0});
while (SZ(q)) {
    auto [u, d] = q.top();
    q.pop();
    if (d != D[u]) continue;
    for (auto [v, w] : G[u]) {
        if (D[v] > d + w) {
            D[v] = d + w;
            q.push({v, D[v]});
        }
    }
}
```

2.2 Tarjan SCC

```
int tim = 0, scc = 0;
vector<int> dfn(n), low(n), bel(n, -1), stk;
auto tarj = [&](auto _, int u) -> void {
    dfn[u] = low[u] = ++tim;
    stk.push_back(u);
    for (int v : G[u]) {
        if (!dfn[v]) {
            _(_, v);
            low[u] = min(low[u], low[v]);
        } else if (bel[v] == -1) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] != low[u]) return;
    while (1) {
        int v = stk.back();
        stk.pop_back();
        bel[v] = scc;
        if (v == u) break;
    }
    scc++;
};
for (int u = 0; u < n; u++) {
    if (!dfn[u]) tarj(tarj, u);
}
```

2.3 2-SAT

- $u_i = 2 \times u + i$
- $u_a \vee v_b$: 加入 $u_a \oplus 1 \rightarrow v_b$ 和 $v_b \oplus 1 \rightarrow u_a$

- $u_a \Rightarrow v_b$: 加入 $u_a \rightarrow v_b$ 和 $v_b \oplus 1 \rightarrow u_a \oplus 1$
- 强制 $u = a$: 加入 $u_a \oplus 1 \rightarrow u_a$
- 禁止 $u = a$ 与 $v = b$ 同时成立: 加入 $u_a \rightarrow v_b \oplus 1$ 和 $v_b \rightarrow u_a \oplus 1$
- $u = v$: 加入 $u_0 \leftrightarrow v_0$ 和 $u_1 \leftrightarrow v_1$
- $u \neq v$: 加入 $u_0 \leftrightarrow v_1$ 和 $u_1 \leftrightarrow v_0$

其中每个 $\leftrightarrow$ 都表示正反两条有向边。

若存在 u 满足 u_0, u_1 在同一强连通分量中, 则无解。

构造: $\text{ans}_u = [\text{bel}_{u_0} > \text{bel}_{u_1}]$ 。

2.4 Segment Tree Graph

- 原图和区间均使用 0-index: 原图节点编号为 $0 \sim n-1$, $\text{mdf}(l, r, \dots)$ 中的 $[l, r]$ 为闭区间。
- ZKW 线段树内部仍使用从 1 开始的堆式编号, 叶子位置为 $S \sim 2S-1$, 原图节点 i 对应叶子 $S+i$ 。
- sn 的值为 $2S-1$, 表示一棵完整 ZKW 线段树的节点数。
- 对于 $1 \leq p \leq \text{sn}$, $\text{id}(p, 0)$ 为 $n+p-1$, 表示 down 树节点; $\text{id}(p, 1)$ 为 $n+\text{sn}+p-1$, 表示 up 树节点。
- tot 初始为 $n+2(2S-1)$, 表示当前节点数, 也是下一个可用节点编号; 初始有效编号为 $0 \sim \text{tot}-1$, $\text{Node}()$ 返回 tot 后将其加一。

```
int S = 1;
while (S < n) S <<= 1;
int sn = 2 * S - 1;
int tot = n + 2 * sn;
vector<vector<pair<int, LL>>> G(tot);
auto id = [&](int p, int up) {
    return n + p - 1 + up * sn;
};
auto add = [&](int u, int v, LL w) {
    G[u].push_back({v, w});
};
auto link = [&](int u, int v, LL w, int up) {
    if (up) swap(u, v);
    add(u, v, w);
};
auto Node = [&]() {
    G.emplace_back();
    return tot++;
};
for (int up = 0; up < 2; up++) {
    for (int p = 1; p < S; p++) {
        link(id(p, up), id(p * 2, up), 0, up);
        link(id(p, up), id(p * 2 + 1, up), 0, up);
    }
    for (int i = 0; i < n; i++) {
```

```

    link(id(S + i, up), i, 0, up);
}
}
auto mdf = [&](int l, int r, int x, LL w, int up) {
    for (l += S, r += S + 1; l < r; l >>= 1, r >>= 1) {
        if (l & 1) link(x, id(l++, up), w, up);
        if (r & 1) link(x, id(--r, up), w, up);
    }
};

```

```

// x -> [l, r]
mdf(l, r, x, w, 0);
// [l, r] -> x
mdf(l, r, x, w, 1);
// [l1, r1] -> [l2, r2]
int x = Node();
mdf(l1, r1, x, 0, 1);
mdf(l2, r2, x, w, 0);

```

3 Data Structure

3.1 SGT

- 所有位置和区间均为 0-index, $\text{prod}(l, r)$ 使用半开区间 $[l, r)$ 。
- $\text{max_right}/\text{min_left}$ 的判定函数须对单位元为真, 并且在查询方向上具有单调性。

```

template <class S, S (*op)(S, S), S (*e)()> struct SGT {
    int n, sz = 1;
    vector<S> t;
    SGT(int n = 0) : SGT(vector<S>(n, e())) {}
    SGT(const vector<S>& a) : n(a.size()) {
        while (sz < n) sz <<= 1;
        t.assign(2 * sz, e());
        for (int i = 0; i < n; i++) t[sz + i] = a[i];
        for (int p = sz - 1; p; p--) upd(p);
    }
    void upd(int p) {
        t[p] = op(t[2 * p], t[2 * p + 1]);
    }
    void set(int p, S x) {
        assert(0 <= p && p < n);
        t[p += sz] = x;
        while (p >>= 1) upd(p);
    }
    S get(int p) {
        assert(0 <= p && p < n);
        return t[p + sz];
    }
    S prod(int l, int r) {

```

```

        assert(0 <= l && l <= r && r <= n);
        S x = e(), y = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1) {
            if (l & 1) x = op(x, t[l++]);
            if (r & 1) y = op(t[--r], y);
        }
        return op(x, y);
    }
    S all_prod() { return t[1]; }
    template <class F> int max_right(int l, F&& f) {
        assert(0 <= l && l <= n && f(e()));
        if (l == n) return n;
        l += sz;
        S x = e();
        do {
            while (!(l & 1)) l >>= 1;
            if (!f(op(x, t[l]))) {
                while (l < sz) {
                    l *= 2;
                    if (f(op(x, t[l]))) x = op(x, t[l++]);
                }
                return l - sz;
            }
        } while ((l & -l) != l);
        return n;
    }
    template <class F> int min_left(int r, F&& f) {
        assert(0 <= r && r <= n && f(e()));
        if (!r) return 0;
        r += sz;
        S x = e();
        do {
            r--;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!f(op(t[r], x))) {
                while (r < sz) {
                    r = 2 * r + 1;
                    if (f(op(t[r], x))) x = op(t[r--], x);
                }
                return r + 1 - sz;
            }
        } while ((r & -r) != r);
        return 0;
    }
};

```

3.2 LazySGT

- 所有位置和区间均为 0-index，区间使用半开形式 $[l, r)$ 。
- $FS(f, x)$ 表示标记 f 作用于信息 x ； $FF(f, g)$ 表示先作用 g 、再作用 f 。
- $\text{max_right}/\text{min_left}$ 的判定函数须对单位元为真，并且在查询方向上具有单调性。

```
template <class S, S (*op)(S, S), S (*e)(),
          class F, S (*FS)(F, S), F (*FF)(F, F), F (*id)()>
struct LazySGT {
    int n, sz, lg = 0;
    vector<S> t;
    vector<F> lz;
    LazySGT(int n = 0) : LazySGT(vector<S>(n, e())) {}
    LazySGT(const vector<S>& a) : n(a.size()) {
        while ((1 << lg) < n) lg++;
        sz = 1 << lg;
        t.assign(2 * sz, e());
        lz.assign(sz, id());
        for (int i = 0; i < n; i++) t[sz + i] = a[i];
        for (int p = sz - 1; p; p--) upd(p);
    }
    void upd(int p) {
        t[p] = op(t[2 * p], t[2 * p + 1]);
    }
    void push(int p, F f) {
        t[p] = FS(f, t[p]);
        if (p < sz) lz[p] = FF(f, lz[p]);
    }
    void down(int p) {
        push(2 * p, lz[p]);
        push(2 * p + 1, lz[p]);
        lz[p] = id();
    }
    void set(int p, S x) {
        assert(0 <= p && p < n);
        p += sz;
        for (int i = lg; i; i--) down(p >> i);
        t[p] = x;
        for (int i = 1; i <= lg; i++) upd(p >> i);
    }
    S get(int p) {
        assert(0 <= p && p < n);
        p += sz;
        for (int i = lg; i; i--) down(p >> i);
        return t[p];
    }
    S prod(int l, int r) {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return e();
        l += sz;

```

```
        r += sz;
        for (int i = lg; i; i--) {
            if (((l >> i) << i) != l) down(l >> i);
            if (((r >> i) << i) != r) down((r - 1) >> i);
        }
        S x = e(), y = e();
        while (l < r) {
            if (l & 1) x = op(x, t[l++]);
            if (r & 1) y = op(t[--r], y);
            l >>= 1;
            r >>= 1;
        }
        return op(x, y);
    }
    S all_prod() { return t[1]; }
    void apply(int p, F f) {
        assert(0 <= p && p < n);
        p += sz;
        for (int i = lg; i; i--) down(p >> i);
        t[p] = FS(f, t[p]);
        for (int i = 1; i <= lg; i++) upd(p >> i);
    }
    void apply(int l, int r, F f) {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return;
        l += sz;
        r += sz;
        for (int i = lg; i; i--) {
            if (((l >> i) << i) != l) down(l >> i);
            if (((r >> i) << i) != r) down((r - 1) >> i);
        }
        int a = l, b = r;
        while (l < r) {
            if (l & 1) push(l++, f);
            if (r & 1) push(--r, f);
            l >>= 1;
            r >>= 1;
        }
        l = a;
        r = b;
        for (int i = 1; i <= lg; i++) {
            if (((l >> i) << i) != l) upd(l >> i);
            if (((r >> i) << i) != r) upd((r - 1) >> i);
        }
    }
    template <class G> int max_right(int l, G&& g) {
        assert(0 <= l && l <= n && g(e()));
        if (l == n) return n;
        l += sz;
        for (int i = lg; i; i--) down(l >> i);

```

```

S x = e();
do {
    while (!(l & 1)) l >>= 1;
    if (!g(op(x, t[l]))) {
        while (l < sz) {
            down(l);
            l *= 2;
            if (g(op(x, t[l]))) x = op(x, t[l++]);
        }
        return l - sz;
    }
    x = op(x, t[l++]);
} while ((l & -l) != l);
return n;
}

template <class G> int min_left(int r, G&& g) {
    assert(0 <= r && r <= n && g(e()));
    if (!r) return 0;
    r += sz;
    for (int i = lg; i; i--) down((r - 1) >> i);
    S x = e();
    do {
        r--;
        while (r > 1 && (r & 1)) r >>= 1;
        if (!g(op(t[r], x))) {
            while (r < sz) {
                down(r);
                r = 2 * r + 1;
                if (g(op(t[r], x))) x = op(t[r--], x);
            }
            return r + 1 - sz;
        }
        x = op(t[r], x);
    } while ((r & -r) != r);
    return 0;
}
};

```

```

struct S {
    LL sum;
    int len;
};

S op(S a, S b) { return {a.sum + b.sum, a.len + b.len}; }
S e() { return {0, 0}; }
using F = LL;
S FS(F f, S x) { return {x.sum + f * x.len, x.len}; }
F FF(F f, F g) { return f + g; }
F id() { return 0; }
using Seg = LazySGT<S, op, e, F, FS, FF, id>;
vector<S> a(n, {0, 1});

```

```

Seg seg(a);
seg.apply(l, r, x);
LL ans = seg.prod(l, r).sum;

```

3.3 Chairman Tree

- 值域使用 $0 \sim m - 1$ ；依次调用 `add(x)` 后，`rt[i]` 表示前 i 个元素形成的版本。
- `count(l, r, ql, qr)` 统计下标区间 $[l, r)$ 中值落在 $[ql, qr)$ 的元素个数。
- `kth(l, r, k)` 查询 $[l, r)$ 的第 k 小值域下标， k 从 0 开始。

```

struct Chairman {
    struct Node {
        int ls, rs, sum;
    };
    int n;
    vector<Node> t = {{0, 0, 0}};
    vector<int> rt = {0};
    Chairman(int n) : n(n) { assert(n > 0); }
    int copy(int p) {
        t.push_back(t[p]);
        return SZ(t) - 1;
    }
    int add(int p, int l, int r, int x, int v) {
        int q = copy(p);
        t[q].sum += v;
        if (r - l == 1) return q;
        int m = (l + r) / 2;
        if (x < m) t[q].ls = add(t[q].ls, l, m, x, v);
        else t[q].rs = add(t[q].rs, m, r, x, v);
        return q;
    }
    void add(int x, int v = 1) {
        assert(0 <= x && x < n);
        rt.push_back(add(rt.back(), 0, n, x, v));
    }
    int count(int a, int b, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[b].sum - t[a].sum;
        int m = (l + r) / 2;
        return count(t[a].ls, t[b].ls, l, m, ql, qr)
            + count(t[a].rs, t[b].rs, m, r, ql, qr);
    }
    int count(int l, int r, int ql, int qr) {
        assert(0 <= l && l <= r && r < SZ(rt));
        assert(0 <= ql && ql <= qr && qr <= n);
        return count(rt[l], rt[r], 0, n, ql, qr);
    }
    int kth(int a, int b, int l, int r, int k) {
        if (r - l == 1) return l;

```

```

    int m = (l + r) / 2;
    int s = t[t[b].ls].sum - t[t[a].ls].sum;
    if (k < s) return kth(t[a].ls, t[b].ls, l, m, k);
    return kth(t[a].rs, t[b].rs, m, r, k - s);
}

int kth(int l, int r, int k) {
    assert(0 <= l && l <= r && r < SZ(rt));
    assert(0 <= k && k < t[rt[r]].sum - t[rt[l]].sum);
    return kth(rt[l], rt[r], 0, n, k);
}
};

```

```

vector<int> xs = a;
sort(ALL(xs));
xs.erase(unique(ALL(xs)), xs.end());
Chairman seg(SZ(xs));
for (int x : a) {
    seg.add(lower_bound(ALL(xs), x) - xs.begin());
}
int p = seg.kth(l, r, k);
int ans = xs[p];

```

3.4 Li Chao Tree

- 维护整数横坐标 $[L, R)$ 上直线的最小值，支持整条直线或限制在 $[l, r)$ 内的线段插入。
- $\text{add}(\{k, b\})$ 插入 $y = kx + b$ ， $\text{qry}(x)$ 查询横坐标 x 处的最小值；求最大值时将直线和答案同时取反。
- 动态开点，空间复杂度与实际插入产生的节点数有关。计算结果会被截断到 $[-4 \times 10^{18}, 4 \times 10^{18}]$ 。

```

struct LiChao {
    static constexpr LL INF = (LL)4e18;
    struct Line {
        LL k, b;
        Line(LL k = 0, LL b = INF) : k(k), b(b) {}
        LL get(LL x) {
            I y = (I)k * x + b;
            if (y > INF) return INF;
            if (y < -INF) return -INF;
            return y;
        }
    };
};

struct Node {
    Line f;
    int ls = 0, rs = 0;
};

int L, R;

```

```

vector<Node> t = {Node(), Node()};
LiChao(int L, int R) : L(L), R(R) { assert(L < R); }
int node() {
    t.emplace_back();
    return SZ(t) - 1;
}

void add(Line f, int p, int l, int r) {
    int m = l + (r - l) / 2;
    bool x = f.get(l) < t[p].f.get(l);
    bool y = f.get(m) < t[p].f.get(m);
    if (y) swap(f, t[p].f);
    if (r - l == 1) return;
    if (x != y) {
        if (!t[p].ls) t[p].ls = node();
        add(f, t[p].ls, l, m);
    } else {
        if (!t[p].rs) t[p].rs = node();
        add(f, t[p].rs, m, r);
    }
}

void add(Line f) { add(f, 1, L, R); }
void add(int ql, int qr, Line f, int p, int l, int r) {
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) return add(f, p, l, r);
    int m = l + (r - l) / 2;
    if (ql < m) {
        if (!t[p].ls) t[p].ls = node();
        add(ql, qr, f, t[p].ls, l, m);
    }
    if (m < qr) {
        if (!t[p].rs) t[p].rs = node();
        add(ql, qr, f, t[p].rs, m, r);
    }
}

void add(int l, int r, Line f) {
    assert(L <= l && l <= r && r <= R);
    if (l < r) add(l, r, f, 1, L, R);
}

LL qry(int x, int p, int l, int r) {
    if (!p) return INF;
    LL ans = t[p].f.get(x);
    if (r - l == 1) return ans;
    int m = l + (r - l) / 2;
    if (x < m) return min(ans, qry(x, t[p].ls, l, m));
    return min(ans, qry(x, t[p].rs, m, r));
}

LL qry(int x) {
    assert(L <= x && x < R);
    return qry(x, 1, L, R);
}

```



```
};
```

3.5 Segment Tree Beats

- 所有位置和区间均为 0-index，区间使用半开形式 $[l, r)$ 。
- 支持区间加、区间 chmin、区间 chmax，以及区间和、最大值、最小值查询。
- chmin/chmax 利用严格次大值和严格次小值剪枝，复杂度为线段树 Beats 的均摊复杂度。

```
struct Beats {
    static constexpr LL INF = (LL)4e18;
    struct Node {
        LL sum = 0, add = 0;
        LL mx = -INF, smx = -INF;
        LL mn = INF, smn = INF;
        int cmx = 0, cmn = 0;
    };
    int n;
    vector<Node> t;
    Beats(const vector<LL>& a) : n(a.size()), t(4 * n) {
        assert(n > 0);
        build(1, 0, n, a);
    }
    Node merge(Node a, Node b) {
        Node c;
        c.sum = a.sum + b.sum;
        if (a.mx == b.mx) {
            c.mx = a.mx;
            c.smx = max(a.smx, b.smx);
            c.cmx = a.cmx + b.cmx;
        } else if (a.mx > b.mx) {
            c.mx = a.mx;
            c.smx = max(a.smx, b.mx);
            c.cmx = a.cmx;
        } else {
            c.mx = b.mx;
            c.smx = max(a.mx, b.smx);
            c.cmx = b.cmx;
        }
        if (a.mn == b.mn) {
            c.mn = a.mn;
            c.smn = min(a.smn, b.smn);
            c.cmn = a.cmn + b.cmn;
        } else if (a.mn < b.mn) {
            c.mn = a.mn;
            c.smn = min(a.smn, b.mn);
            c.cmn = a.cmn;
        } else {
            c.mn = b.mn;

```

```
            c.smn = min(a.smn, b.smn);
            c.cmn = b.cmn;
        }
        return c;
    }
    void build(int p, int l, int r, const vector<LL>& a) {
        if (r - l == 1) {
            t[p].sum = t[p].mx = t[p].mn = a[l];
            t[p].cmx = t[p].cmn = 1;
            return;
        }
        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m, r, a);
        t[p] = merge(t[2 * p], t[2 * p + 1]);
    }
    void apply_add(int p, int len, LL x) {
        t[p].sum += x * len;
        t[p].mx += x;
        t[p].mn += x;
        if (t[p].smx != -INF) t[p].smx += x;
        if (t[p].smn != INF) t[p].smn += x;
        t[p].add += x;
    }
    void apply_chmin(int p, LL x) {
        if (t[p].mx <= x) return;
        t[p].sum += (x - t[p].mx) * t[p].cmx;
        if (t[p].mn == t[p].mx) t[p].mn = x;
        else if (t[p].smn == t[p].mx) t[p].smn = x;
        t[p].mx = x;
    }
    void apply_chmax(int p, LL x) {
        if (t[p].mn >= x) return;
        t[p].sum += (x - t[p].mn) * t[p].cmn;
        if (t[p].mx == t[p].mn) t[p].mx = x;
        else if (t[p].smx == t[p].mn) t[p].smx = x;
        t[p].mn = x;
    }
    void down(int p, int l, int r) {
        int m = (l + r) / 2;
        if (t[p].add) {
            apply_add(2 * p, m - l, t[p].add);
            apply_add(2 * p + 1, r - m, t[p].add);
            t[p].add = 0;
        }
        if (t[2 * p].mx > t[p].mx) apply_chmin(2 * p, t[p].mx);
        if (t[2 * p + 1].mx > t[p].mx) apply_chmin(2 * p + 1, t[p].mx);
        if (t[2 * p].mn < t[p].mn) apply_chmax(2 * p, t[p].mn);
        if (t[2 * p + 1].mn < t[p].mn) apply_chmax(2 * p + 1, t[p].mn);
    }
}
```

```

void up(int p) {
    t[p] = merge(t[2 * p], t[2 * p + 1]);
}
void add(int ql, int qr, LL x, int p, int l, int r) {
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) return apply_add(p, r - l, x);
    down(p, l, r);
    int m = (l + r) / 2;
    add(ql, qr, x, 2 * p, l, m);
    add(ql, qr, x, 2 * p + 1, m, r);
    up(p);
}
void chmin(int ql, int qr, LL x, int p, int l, int r) {
    if (qr <= l || r <= ql || t[p].mx <= x) return;
    if (ql <= l && r <= qr && t[p].smx < x) {
        return apply_chmin(p, x);
    }
    down(p, l, r);
    int m = (l + r) / 2;
    chmin(ql, qr, x, 2 * p, l, m);
    chmin(ql, qr, x, 2 * p + 1, m, r);
    up(p);
}
void chmax(int ql, int qr, LL x, int p, int l, int r) {
    if (qr <= l || r <= ql || t[p].mn >= x) return;
    if (ql <= l && r <= qr && t[p].smn > x) {
        return apply_chmax(p, x);
    }
    down(p, l, r);
    int m = (l + r) / 2;
    chmax(ql, qr, x, 2 * p, l, m);
    chmax(ql, qr, x, 2 * p + 1, m, r);
    up(p);
}
LL qsum(int ql, int qr, int p, int l, int r) {
    if (qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return t[p].sum;
    down(p, l, r);
    int m = (l + r) / 2;
    return qsum(ql, qr, 2 * p, l, m)
        + qsum(ql, qr, 2 * p + 1, m, r);
}
LL qmax(int ql, int qr, int p, int l, int r) {
    if (qr <= l || r <= ql) return -INF;
    if (ql <= l && r <= qr) return t[p].mx;
    down(p, l, r);
    int m = (l + r) / 2;
    return max(qmax(ql, qr, 2 * p, l, m),
        qmax(ql, qr, 2 * p + 1, m, r));
}

```

```

LL qmin(int ql, int qr, int p, int l, int r) {
    if (qr <= l || r <= ql) return INF;
    if (ql <= l && r <= qr) return t[p].mn;
    down(p, l, r);
    int m = (l + r) / 2;
    return min(qmin(ql, qr, 2 * p, l, m),
        qmin(ql, qr, 2 * p + 1, m, r));
}
void add(int l, int r, LL x) {
    assert(0 <= l && l <= r && r <= n);
    if (l < r) add(l, r, x, 1, 0, n);
}
void chmin(int l, int r, LL x) {
    assert(0 <= l && l <= r && r <= n);
    if (l < r) chmin(l, r, x, 1, 0, n);
}
void chmax(int l, int r, LL x) {
    assert(0 <= l && l <= r && r <= n);
    if (l < r) chmax(l, r, x, 1, 0, n);
}
LL qsum(int l, int r) {
    assert(0 <= l && l < r && r <= n);
    return qsum(l, r, 1, 0, n);
}
LL qmax(int l, int r) {
    assert(0 <= l && l < r && r <= n);
    return qmax(l, r, 1, 0, n);
}
LL qmin(int l, int r) {
    assert(0 <= l && l < r && r <= n);
    return qmin(l, r, 1, 0, n);
}
};

```

3.6 Segment Tree Merge / Split

- 动态开点维护值域 $[0, n)$ 上的非负计数；根节点用一个外部 `int` 保存，空树根为 `0`。
- `merge(a, b)` 将两棵树破坏性合并并返回新根；`split(p, l, r)` 从 `p` 中取走 $[l, r)$ 并返回取出的根。
- `kth(p, k)` 查询第 k 小值域下标， k 从 `0` 开始。若需支持该操作，应保证所有叶子计数非负。

```

struct SplitSGT {
    struct Node {
        int ls = 0, rs = 0;
        LL sum = 0;
    };
    int n;
    vector<Node> t = {Node()};
}

```

```

SplitSGT(int n) : n(n) { assert(n > 0); }
int node() {
    t.emplace_back();
    return SZ(t) - 1;
}
void up(int p) {
    t[p].sum = t[t[p].ls].sum + t[t[p].rs].sum;
}
int add(int p, int l, int r, int x, LL v) {
    if (!p) p = node();
    if (r - l == 1) {
        t[p].sum += v;
        return p;
    }
    int m = (l + r) / 2;
    if (x < m) t[p].ls = add(t[p].ls, l, m, x, v);
    else t[p].rs = add(t[p].rs, m, r, x, v);
    up(p);
    return p;
}
void add(int& p, int x, LL v) {
    assert(0 <= x && x < n);
    p = add(p, 0, n, x, v);
}
LL sum(int p, int l, int r, int ql, int qr) {
    if (!p || qr <= l || r <= ql) return 0;
    if (ql <= l && r <= qr) return t[p].sum;
    int m = (l + r) / 2;
    return sum(t[p].ls, l, m, ql, qr)
        + sum(t[p].rs, m, r, ql, qr);
}
LL sum(int p, int l, int r) {
    assert(0 <= l && l <= r && r <= n);
    return sum(p, 0, n, l, r);
}
int merge(int a, int b, int l, int r) {
    if (!a || !b) return a + b;
    if (r - l == 1) {
        t[a].sum += t[b].sum;
        return a;
    }
    int m = (l + r) / 2;
    t[a].ls = merge(t[a].ls, t[b].ls, l, m);
    t[a].rs = merge(t[a].rs, t[b].rs, m, r);
    up(a);
    return a;
}
int merge(int a, int b) { return merge(a, b, 0, n); }
pair<int, int> split(int p, int l, int r, int ql, int qr) {
    if (!p || qr <= l || r <= ql) return {p, 0};

```

```

    if (ql <= l && r <= qr) return {0, p};
    int m = (l + r) / 2;
    auto [al, bl] = split(t[p].ls, l, m, ql, qr);
    auto [ar, br] = split(t[p].rs, m, r, ql, qr);
    t[p].ls = al;
    t[p].rs = ar;
    up(p);
    if (!al && !ar) p = 0;
    if (!bl && !br) return {p, 0};
    int q = node();
    t[q].ls = bl;
    t[q].rs = br;
    up(q);
    return {p, q};
}
int split(int& p, int l, int r) {
    assert(0 <= l && l <= r && r <= n);
    auto [a, b] = split(p, 0, n, l, r);
    p = a;
    return b;
}
int kth(int p, LL k) {
    assert(p && 0 <= k && k < t[p].sum);
    int l = 0, r = n;
    while (r - l > 1) {
        int m = (l + r) / 2;
        LL s = t[t[p].ls].sum;
        if (k < s) {
            p = t[p].ls;
            r = m;
        } else {
            k -= s;
            p = t[p].rs;
            l = m;
        }
    }
    return l;
}
};

```

4 String

4.1 KMP

- $kmp(s)[i]$ 表示 $s[0, i)$ 的最长相等真前后缀长度，返回数组长度为 $n + 1$ 。
- 模板只依赖 `size()` 和元素判等，可直接用于字符串或一般序列。

```

template <class T> vector<int> kmp(const T& s) {
    int n = s.size();

```

```
vector<int> f(n + 1);
for (int i = 1, j = 0; i < n; i++) {
    while (j && s[i] != s[j]) j = f[j];
    if (s[i] == s[j]) j++;
    f[i + 1] = j;
}
return f;
}
```

4.2 Z Function

- ZFunc(s)[i] 表示 s 与后缀 $s[i, n)$ 的最长公共前缀长度，并令 $z[0]=n$ 。
- exKMP(s, t, z) 返回每个后缀 $s[i, n)$ 与模式串 t 的最长公共前缀长度，其中 $z=ZFunc(s \rightarrow t)$ 。

```
template <class T> vector<int> ZFunc(const T& s) {
    int n = s.size(), l = 0, r = 0;
    vector<int> z(n);
    if (!n) return z;
    z[0] = n;
    for (int i = 1; i < n; i++) {
        if (i <= r && z[i - l] < r - i + 1) z[i] = z[i - l];
        else {
            z[i] = max(0, r - i + 1);
            while (i + z[i] < n && s[i + z[i]] == s[z[i]]) z[i]++;
            if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
        }
    }
    return z;
}

template <class T>
vector<int> exKMP(const T& s, const T& t, const vector<int>& z) {
    int n = s.size(), m = t.size(), l = 0, r = 0;
    vector<int> p(n);
    if (!n) return p;
    while (p[0] < n && p[0] < m && s[p[0]] == t[p[0]]) p[0]++;
    for (int i = 1; i < n; i++) {
        if (i <= r && z[i - l] < r - i + 1) p[i] = z[i - l];
        else {
            p[i] = max(0, r - i + 1);
            while (i + p[i] < n && p[i] < m && s[i + p[i]] == t[p[i]]) p[i]++;
            if (i + p[i] - 1 > r) l = i, r = i + p[i] - 1;
        }
    }
    return p;
}
```

4.3 String Hash

- 双模 Hash，所有位置均为 0-index，get(l, r) 返回半开区间 $s[l, r)$ 的归一化 Hash。
- 两段字符串长度相同且 Hash 相等时可认为相等；Hash 算法始终存在极小碰撞概率。

```
struct StringHash {
    using H = pair<LL, LL>;
    static constexpr LL P1 = 2034452107, P2 = 2013074419;
    static constexpr LL B1 = 137, B2 = 149;
    int n;
    vector<LL> pw1, pw2, h1, h2;
    StringHash(const string& s) : n(s.size()),
        pw1(n + 1, 1), pw2(n + 1, 1), h1(n + 1), h2(n + 1) {
        for (int i = 0; i < n; i++) {
            int x = (unsigned char)s[i] + 1;
            pw1[i + 1] = pw1[i] * B1 % P1;
            pw2[i + 1] = pw2[i] * B2 % P2;
            h1[i + 1] = (h1[i] * B1 + x) % P1;
            h2[i + 1] = (h2[i] * B2 + x) % P2;
        }
    }
    H get(int l, int r) {
        assert(0 <= l && l <= r && r <= n);
        LL x = (h1[r] - h1[l] * pw1[r - l] % P1 + P1) % P1;
        LL y = (h2[r] - h2[l] * pw2[r - l] % P2 + P2) % P2;
        return {x, y};
    }
};
```

4.4 Manacher

- 算法在变换串 $\#s[0]\#s[1]\#\dots\#$ 上运行， $p[i]-1$ 是以变换串位置 i 为中心的最长回文子串在原串中的长度。
- 对应原串半开区间为 $[(i-p[i]+1)/2, (i+p[i]-1)/2)$ 。

```
vector<int> Manacher(const string& s) {
    string t = "#";
    for (char c : s) t += c, t += '#';
    int n = t.size();
    vector<int> p(n);
    for (int i = 0, l = 0, r = 0; i < n; i++) {
        p[i] = i < r ? min(p[2 * l - i], r - i) : 0;
        while (i + p[i] < n && i - p[i] >= 0
            && t[i + p[i]] == t[i - p[i]]) p[i]++;
        if (i + p[i] > r) l = i, r = i + p[i];
    }
    return p;
}
```

4.5 Aho-Corasick

- 字符集固定为小写字母；先用 `add(s)` 插入全部模式串并保存返回的终止节点，再调用一次 `build()`。
- `count(s)[p]` 是节点 p 对应模式串在文本 s 中的出现次数，重复模式串会共享终止节点。

```
struct ACAM {
    struct Node {
        array<int, 26> ch{};
        int fail = 0;
    };
    vector<Node> t = {};
    vector<int> ord;
    ACAM(int n = 0) { t.reserve(n + 1); }
    int add(const string& s) {
        int p = 0;
        for (char x : s) {
            int c = x - 'a';
            assert(0 <= c && c < 26);
            if (!t[p].ch[c]) {
                t[p].ch[c] = t.size();
                t.push_back({});
            }
            p = t[p].ch[c];
        }
        return p;
    }
    void build() {
        queue<int> q;
        ord.clear();
        for (int c = 0; c < 26; c++) {
            if (t[0].ch[c]) q.push(t[0].ch[c]);
        }
        while (SZ(q)) {
            int u = q.front();
            q.pop();
            ord.pb(u);
            for (int c = 0; c < 26; c++) {
                int v = t[u].ch[c];
                if (v) {
                    t[v].fail = t[t[u].fail].ch[c];
                    q.push(v);
                } else {
                    t[u].ch[c] = t[t[u].fail].ch[c];
                }
            }
        }
    }
    vector<LL> count(const string& s) {
```

```
vector<LL> cnt(t.size());
int p = 0;
for (char x : s) {
    int c = x - 'a';
    assert(0 <= c && c < 26);
    p = t[p].ch[c];
    cnt[p]++;
}
for (int i = SZ(ord) - 1; i >= 0; i--) {
    int u = ord[i];
    cnt[t[u].fail] += cnt[u];
}
return cnt;
}
};
```

4.6 Suffix Array

- `sa[i]` 是排名第 i 的后缀起点，`rk[i]` 是后缀 $s[i, n)$ 的排名，均为 0-index。
- `lc[i]` 等于 `lcp(s[sa[i], n), s[sa[i+1], n])`，数组长度为 $n - 1$ 。

```
struct SuffixArray {
    int n;
    vector<int> sa, rk, lc;
    SuffixArray(const string& s) : n(s.size()), sa(n), rk(n), lc(max(0, n - 1)) {
        if (!n) return;
        iota(ALL(sa), 0);
        sort(ALL(sa), [&](int a, int b) { return s[a] < s[b]; });
        rk[sa[0]] = 0;
        for (int i = 1; i < n; i++) {
            rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
        }
        vector<int> tmp, cnt(n);
        tmp.reserve(n);
        for (int w = 1; rk[sa[n - 1]] < n - 1; w *= 2) {
            tmp.clear();
            for (int i = max(0, n - w); i < n; i++) tmp.pb(i);
            for (int i = 0; i < n; i++) if (sa[i] >= w) tmp.pb(sa[i] - w);
            fill(ALL(cnt), 0);
            for (int i = 0; i < n; i++) cnt[rk[i]]++;
            partial_sum(ALL(cnt), cnt.begin());
            for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[tmp[i]]]] = tmp[i];
            swap(rk, tmp);
            rk[sa[0]] = 0;
            for (int i = 1; i < n; i++) {
                int a = sa[i - 1], b = sa[i];
                int x = a + w < n ? tmp[a + w] : -1;
                int y = b + w < n ? tmp[b + w] : -1;
                rk[b] = rk[a] + (tmp[a] != tmp[b] || x != y);
            }
        }
    }
};
```

```

    }
}
for (int i = 0, k = 0; i < n; i++) {
    int j = rk[i];
    if (!j) {
        k = 0;
        continue;
    }
    if (k) k--;
    int p = sa[j - 1];
    while (i + k < n && p + k < n && s[i + k] == s[p + k]) k++;
    lc[j - 1] = k;
}
}
};

```

4.7 Suffix Automaton

- 根节点编号为 1；普通字符串从 $p=1$ 开始，依次令 $p=\text{sam.ins}(p, c-\text{'a'})$ 。
- 最多产生 $2n$ 个节点； fa 是后缀链接， len 是状态所表示子串的最大长度。

```

template <int C = 26>
struct SAM {
    vector<array<int, C>> ch;
    vector<int> fa, len;
    SAM(int n = 0) : ch(2), fa(2), len(2) {
        ch.reserve(2 * n + 2);
        fa.reserve(2 * n + 2);
        len.reserve(2 * n + 2);
    }
    int node() {
        ch.push_back({});
        fa.pb(0);
        len.pb(0);
        return SZ(ch) - 1;
    }
    int ins(int p, int c) {
        assert(0 <= c && c < C);
        int cur = node();
        len[cur] = len[p] + 1;
        for (; p && !ch[p][c]; p = fa[p]) ch[p][c] = cur;
        if (!p) fa[cur] = 1;
        else {
            int q = ch[p][c];
            if (len[q] == len[p] + 1) fa[cur] = q;
            else {
                int cl = node();
                len[cl] = len[p] + 1;
                ch[cl] = ch[q];

```

```

                fa[cl] = fa[q];
                fa[q] = fa[cur] = cl;
                for (; p && ch[p][c] == q; p = fa[p]) ch[p][c] = cl;
            }
        }
        return cur;
    }
};

```

4.8 Palindromic Tree

- 字符须先映射到 $[0, C)$ ；调用 $\text{init}(n)$ 后逐个 $\text{add}(c)$ ，节点 0、1 分别是长度 0、-1 的根。
- $\text{size}()$ 是本质不同回文串数， $\text{suf_cnt}()$ 是当前回文后缀数， $\text{count}()$ 是已经插入串的回文子串总数。
- 全部插入后调用一次 $\text{calc}()$ ，即可由 $t[p].\text{cnt}$ 得到每个回文串的出现次数。

```

template <int C = 26>
struct PAM {
    struct Node {
        array<int, C> ch{};
        int fail = 0, len = 0, cnt = 0, num = 0;
    };
    vector<Node> t;
    vector<int> s;
    int las;
    LL tot;
    void init(int n) {
        t.assign(2, {});
        t.reserve(n + 2);
        t[0].fail = t[1].fail = 1;
        t[1].len = -1;
        s.assign(1, -1);
        s.reserve(n + 1);
        las = 0;
        tot = 0;
    }
    int get_fail(int p, int c) {
        int n = SZ(s) - 1;
        while (s[n - t[p].len - 1] != c) p = t[p].fail;
        return p;
    }
    int add(int c) {
        assert(0 <= c && c < C);
        s.pb(c);
        int p = get_fail(las, c);
        if (!t[p].ch[c]) {
            int q = get_fail(t[p].fail, c);
            Node x;

```

```

    x.len = t[p].len + 2;
    x.fail = t[q].ch[c];
    x.num = t[x.fail].num + 1;
    t[p].ch[c] = SZ(t);
    t.pb(x);
}
las = t[p].ch[c];
t[las].cnt++;
tot += t[las].num;
return las;
}
int size() { return SZ(t) - 2; }
int suf_cnt() { return t[las].num; }
LL count() { return tot; }
void calc() {
    for (int p = SZ(t) - 1; p >= 2; p--) t[t[p].fail].cnt += t[p].cnt;
}
};

```

4.9 Lyndon / Minimum Representation

- Duval(s) 返回 Lyndon 分解的 0-index 半开区间，按顺序拼接后恰为原序列。
- min_rep(s) 返回循环同构序列中字典序最小表示的起点；空序列返回 0。

```

template <class T>
vector<pair<int, int>> Duval(const T& s) {
    int n = s.size(), i = 0;
    vector<pair<int, int>> a;
    while (i < n) {
        int j = i, k = i + 1;
        while (k < n && s[j] <= s[k]) {
            if (s[j] < s[k]) j = i;
            else j++;
            k++;
        }
        while (i <= j) {
            int r = i + k - j;
            a.push_back({i, r});
            i = r;
        }
    }
    return a;
}

template <class T> int min_rep(const T& s) {
    int n = s.size(), i = 0, j = 1, k = 0;
    if (!n) return 0;
    while (i < n && j < n && k < n) {
        if (s[(i + k) % n] == s[(j + k) % n]) k++;
        else {

```

```

            if (s[(i + k) % n] > s[(j + k) % n]) i += k + 1;
            else j += k + 1;
            if (i == j) j++;
            k = 0;
        }
    }
    return min(i, j);
}

```

5 Geometry

5.1 vector

```

using db = long double;
struct p2 {
    db x, y;
    db len() { return hypot(x, y); }
    p2 operator+(p2 b) { return {x + b.x, y + b.y}; }
    p2 operator-(p2 b) { return {x - b.x, y - b.y}; }
    p2 operator*(db k) { return {x * k, y * k}; }
    p2 operator/(db k) { return {x / k, y / k}; }
    db operator*(p2 b) { return x * b.x + y * b.y; }
    db operator%(p2 b) { return x * b.y - y * b.x; }
};
using ps = vector<p2>;

```

5.2 seg_poly

```

// pa 到 pb 的叉积，逆时针转为正
db side(p2 p, p2 a, p2 b) { return (a - p) % (b - p); }
db tol(p2 p, p2 a, p2 b) { return abs(side(p, a, b)) / (b - a).len(); }

```